智研 • AI 考研备考助手 — 技术文档

> 参赛项目：全国高校 AI Web 开发挑战赛（主题「智启未来：AI 驱动的 Web 应用创新挑战」） 一句话定位：面向在校大学生（大二~大四）的考研备考助手，帮考生搞清「**该不该考、考什么、怎么考**」，并输出可执行的备考路径。

一、项目概述

1.1 背景与问题

考研备考的典型困境不是"不努力"，而是**信息不对称与计划失焦**：

- **不懂流程**：报名、确认、初试、复试、调剂的时间节点与规则，学硕与专硕的区别，A区/B区、自划线等名词，构成了一道信息门槛；
- **不知道自己哪不会**："感觉复习得不好"是模糊感受，缺乏对具体知识点掌握度的量化认知；
- **计划做不出、执行不下去**：网上的计划模板是给"平均人"的，与个人基础、目标院校、每日可投入时间不匹配。

1.2 解决方案

本项目用 AI 打通三条链路，形成闭环：

环节	解决的问题	做法
认知科普	不懂流程和名词	左侧固定科普文本（术语词典 / 学硕专硕 / 时间线等 7 个主题）+ 右侧基于自建知识库的 AI 答疑（RAG，答案可溯源）
学情诊断	不知道自己哪不会	三科（数学/英语/政治）题库 + 测评 + BKT 掌握度建模，把"感觉不行"变成"具体哪章不行"
计划与执行	计划做不出、执行不下去	掌握度驱动计划权重；按实际进度动态调整；AI 规划助手把"目标一句话"变成"分步执行方案"

1.3 核心设计原则

事实与算法放在代码里（可复现、可测试、可验证），大模型只负责组织语言与个性化推理。 这条原则贯穿整个项目，是避免 AI 幻觉的根本手段：

- 院校层次、专业设置、考试科目等事实全部来自数据库，不经过模型生成；
- 知识点掌握度由 BKT 算法（纯 Java 实现）计算，不经过模型估计；
- 大模型仅承担三类工作：语言组织（把检索结果组织成连贯文章）、个性化润色（把规则计划改写成个人建议）、多轮对话（带历史上下文的答疑）；
- 所有 AI 调用点均有**规则兜底**：模型不可用（限流、故障）时功能降级但不中断，且明确告知用户。

二、需求分析

2.1 目标用户

- **主体**：在校大学生（大二~大四），有考研意向或正在备考；
- **特征**：信息获取能力有限、备考规划经验不足、需要个性化指导；对 AI 工具接受度高。

2.2 功能需求

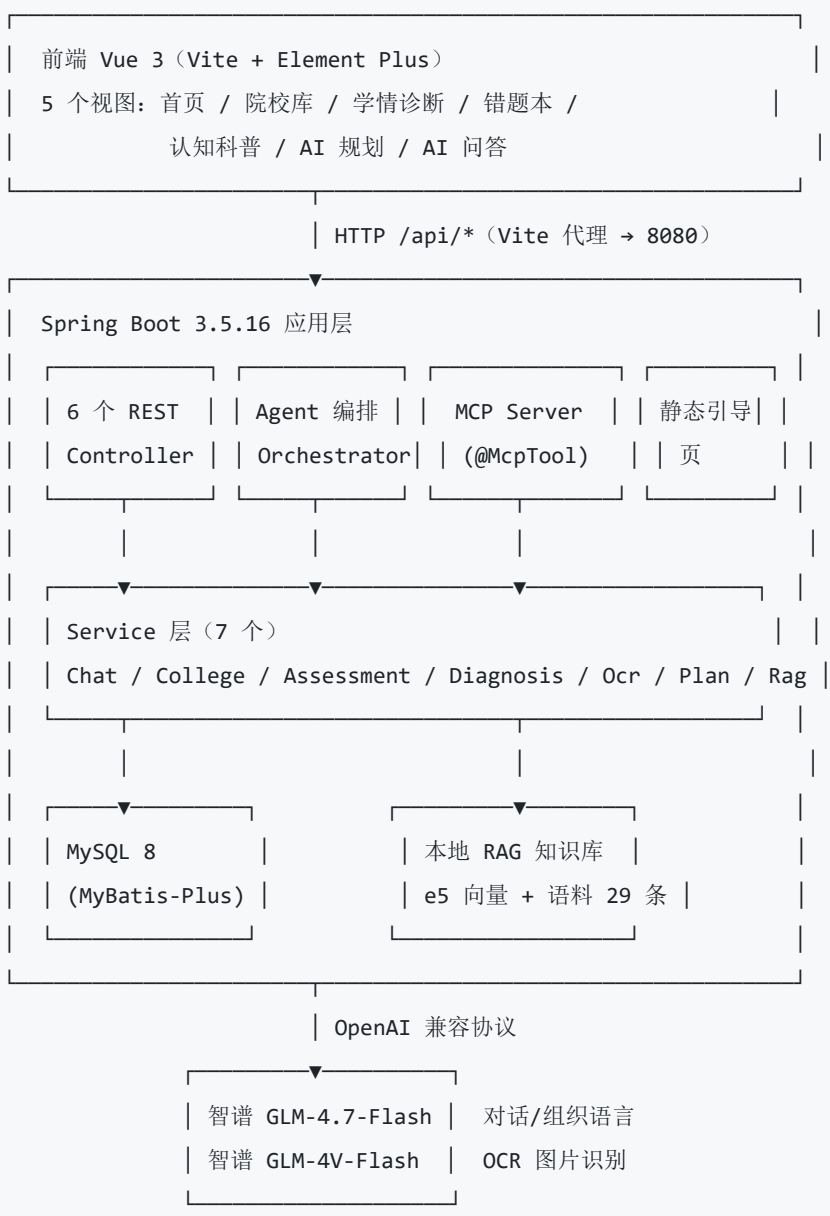
模块	需求	对应实现
院校专业查询	查山东地区 20~30 所院校的热门考研专业、考试科目、学制	院校专业库（28 所 / 104 专业，结构化数据，不收录分数线）
认知科普	系统了解考研流程、时间线、名词、学硕专硕区别	固定科普文本（7 主题）+ RAG 答疑
学情诊断	量化各知识点掌握度，定位薄弱点	三科题库（数学 49 / 英语 37 / 政治 34 题，共 120 道）+ BKT 掌握度建模
错题本	诊断答错自动收录 + 用户导入 + 随机重做	MySQL 错题表 + 随机抽题 + 判分引擎
AI 规划助手	输入一个目标，自动拆步骤执行并给出可执行方案	自研 Agent 编排（5 工具 + 透明执行轨迹）
图片作答	填空题答案含特殊符号不便打字时，可拍照/截图提交	OCR 识别 + 模型判分（GLM-4V-Flash）

2.3 非功能需求

项	要求	落实
无障碍	WCAG 2.1 AA	语义化标签、`aria-label`、`role="log"` + `aria-live` 播报、`prefers-reduced-motion` 尊重、键盘可达
响应式	移动端可用	720px 断点自适应；认知科普页窄屏自动上下堆叠
可用性	AI 故障不拖垮核心功能	全链路规则兜底 + 降级提示
合规	数据与内容合规	题库全部原创不照搬真题；不收录分数线；不抓取受限站点；密钥走环境变量
零门槛	不依赖付费/注册服务	本地 ONNX Embedding + 本地向量库，RAG 链路无需任何 API Key

三、系统架构

3.1 总体架构



3.2 技术选型

层	选型	版本	选型理由
运行时	Java	17 (LTS)	Spring Boot 3.x 基线，长期支持
主框架	Spring Boot	3.5.16	自带 Web 容器，无需额外部署；自动配置生态成熟
AI 框架	Spring AI	1.1.8	**换模型只改一行配置**；自带 RAG / 对话记忆 / MCP 扩展点，避免手写协议
大模型	智谱 GLM-4.7-Flash	免费档	国内直连、200K 上下文、兼容 OpenAI 协议、免费档足够演示
视觉模型	智谱 GLM-4V-Flash	免费档	与文本模型同密钥同端点，专用于 OCR 图片作答

层	选型	版本	选型理由
Embedding	e5-small-v2（本地 ONNX）	—	多语言、对中文友好；**本地推理，无需 API Key、无调用费用、无网络依赖**
向量库	SimpleVectorStore（文件持久化）	—	与 Spring AI 同源；JSON 文件持久化，重启不丢，零运维
数据层	MySQL + MyBatis-Plus	8.0 / 3.5.17	关系数据；LambdaQueryWrapper 少写 SQL
前端	Vue 3 + Vite 6 + Element Plus	—	组合式 API 简洁；Vite 秒级热更新；Element Plus 组件专业统一
接口文档	springdoc-openapi	2.9.1	自动生成 Swagger 页面

> 版本已锁定，不升级 Spring AI 2.x（里程碑版，包名有变动）。

3.3 分层职责

```
Controller 层   6 个：参数校验、权限（CORS）、返回视图对象，不含业务逻辑
Service 层      7 个：业务编排；算法（BKT、计划引擎）为纯 Java，不依赖模型
Agent 层        9 个类：AgentTool 接口 + 5 个工具 + Orchestrator + Context/Result
MCP 层          1 个类：@McpTool 注解暴露 5 个标准工具，starter 自动扫描
Config 层       4 个：RagConfig（VectorStore 装配）、三个数据初始化器
```

四、AI 技术方案（重点）

本章按赛事评分要求，阐述大模型集成、Agent 工作流、RAG、多模态、AI 安全可控、MCP 工具集成六个方向的**选型理由、集成方式与优化策略**。

4.1 大模型集成：GLM-4.7-Flash

选型理由：国内直连（无跨境网络问题）、200K 长上下文（可携带完整诊断材料）、兼容 OpenAI 协议（Spring AI 一套代码接所有模型）、免费档（学生项目零成本）。

集成方式：通过 Spring AI 的 OpenAI 兼容客户端接入，配置仅三行（`base-url` / `api-key` / `chat model`），`ChatClient.Builder` 统一构建；密钥走环境变量 `ZHIPU_API_KEY`，代码中只有 `${ZHIPU_API_KEY}` 占位符。

优化策略：

1. **提示词工程**：所有 AI 调用都带"只基于给定信息回答，不编造任何数据"的系统指令，材料以结构化文本（【资料】分块）注入，降低幻觉；
2. **上下文预算**：诊断材料按需裁剪（掌握度只取薄弱项），控制 token 消耗；
3. **优雅降级**：每个调用点 try-catch，失败走规则模板兜底（详见 4.6）。

> 排查线上 500 时发现 `ChatService` 曾是全项目唯一漏掉 try-catch 的 AI 调用点——免费档模型一限流，AI 问答接口就返回 500。已修复并补哨兵测试（`ChatServiceTest`），现在**全部 6 类 AI 调用（对话/流式/RAG 问答/讲解/Agent 汇总/计划润色）均有兜底。**

落地场景（4 处，全部带兜底）：科普讲解合成、AI 答疑多轮对话、Agent 汇总润色。

4.2 RAG 认知科普：本地 Embedding + SimpleVectorStore

选型理由：RAG 需要"检索 + 生成"两半。检索半边如果用云端 Embedding API，会产生费用且受网络影响；本项目选用**本地 ONNX 推理的 e5-small-v2**（多语言模型，中文效果好），配合 Spring AI 的 SimpleVectorStore（JSON 文件持久化），实现**零 API Key、零费用、零额外运维**的完整 RAG 链路。生成半边仍用 GLM-4.7-Flash（限流时降级为原文拼接）。

架构与数据流：

```
kaoyan-corpus.json（29 条结构化事实：流程/名词/学硕专硕/时间线/科目/院校/建议）
        | CorpusInitializer（@PostConstruct，幂等）
        ▼
SimpleVectorStore（kaoyan-vectorstore.json，启动时构建/加载）
        | RagService.ask(question)
        ▼
topK=4 相似度检索 → 阈值过滤（无命中即回答"没有相关信息"，不编造）
        |
        ▼
GLM 组织成连贯回答 + 返回 sources（参考来源，答案可溯源）
```

集成方式：

- `spring-ai-starter-model-transformers` + `spring-ai-vector-store`（均为 1.1.8 BOM 托管坐标）；
- `RagConfig` 显式注入 `TransformersEmbeddingModel` 构建 `SimpleVectorStore` Bean（消歧义）；
- 检索与生成分离：`RagService` 负责检索与提示词组装，模型只做最后一步语言组织。

优化策略与踩坑（重要）：

1. **分词器配置节点**：Spring AI 1.1.8 中分词器属性位于**独立节点** `spring.ai.embedding.transformer.tokenizer.uri`，不在 `onnx` 下。若误写在 `onnx.tokenizer-uri`，**不会报错**，但会静默回退到框架默认的 all-MiniLM 分词器——与 e5 模型不匹配导致向量失真、检索失效。这是开发中实测定位并修复的真 bug（见 6.3）；
2. **防幻觉三段式**：检索阈值过滤（低于阈值不调用模型，直接回答"知识库没有相关信息"）→ 提示词限定"只依据资料作答" → 返回 `sources` 供用户核对；
3. **降级链**：模型限流 → 返回兜底文案 + 已检索来源，HTTP 仍 200；
4. **缓存目录固定 + 模型本地化**：Spring AI 1.1.8 的 transformers 缓存默认是每次 JVM 启动新建的随机临时目录（`Files.createTempDirectory("spring-ai-model-cache")`，反编译 jar 实锤）——会导致「每次重启都重新下载 100MB 模型」且临时目录永不清理。已通过 `spring.ai.embedding.transformer.cache.directory=${user.home}/.aiweb/models` 钉到固定位置；进一步地，因 hf-mirror 的 model.onnx 302 跳转 CDN 在国内网络下极慢（实测 20 秒 1KB，启动必卡死），模型文件已本地化到 `~/.aiweb/models/e5/`（yml 直接 `file:///` 读），新电脑运行 `scripts/download-model.bat` 一键就位；
5. **嵌入向量 L2 归一化**：e5 官方用法要求归一化，但 ONNX 模型与 Spring AI 都不做（反编译字节码确认 mean pooling 后直接输出）——实测范数 7.94，余弦相似度被共模分量主导、全糊在 0.86+，检索排序随机。自研 `NormalizingEmbeddingModel` 装饰器统一归一化（语料构建与检索两侧都经过），分数区分度恢复正常；
6. **e5 前后缀与标题拼接**：按官方用法给 passage 加 `passage:`、query 加 `query:` 前缀；嵌入文本拼接「标题+正文」——标题信息密度高且含关键词（只嵌正文时用户短问「学硕专硕区别」被「数学一二三的区别」挤

掉）；

7. **检索质量**：语料按"一问一事实"粒度拆分（29 条），避免大段文本稀释相似度；科普主题用专题文章保证系统性。

4.3 Agent 工作流：自研 Orchestrator

选型理由：AI 规划助手需要"理解目标 → 决定调哪些能力 → 按序执行 → 汇总"的多步编排。引入 LangGraph 类框架会带来额外概念与依赖；本项目工具数量固定（5 个）、流程线性，**自研轻量 Orchestrator**（约 120 行）更可控、好排查，也更能体现对 Agent 模式（规划-执行-观察）的理解。

架构：



执行轨迹透明化：每一步记录 `{index, name, status(done/skipped), durationMs, output}`，前端以时间线**逐条播放**展示——用户能清楚看到 AI "怎么得出结论"，这是交互创新点，也是 AI 可信度的重要组成。

优化策略：

- 意图判断**不经过模型**（数据驱动 + 关键词），稳定、可复现、零延迟、零费用；
- 每个工具实现统一的 `AgentTool` 接口（`name/order/shouldRun/execute`），新增能力即新增一个工具类，Orchestrator 无需改动；
- `AgentContext` 作为跨步骤共享载体，测试可单独构造；
- 全程规则兜底：任一步骤异常不影响其他步骤，最终汇总失败时用模板组织各步产出。

4.4 多模态：OCR 图片作答

选型理由：数学填空题答案常含特殊符号（根号、分数线、π），手机打字不便。GLM-4.7-Flash 为纯文本模型，而同密钥同端点的 **GLM-4V-Flash（免费视觉模型）**可直接理解图片，无需新增依赖与密钥。

集成方式：`OcrService` 将前端压缩后的图片（canvas 压至 1000px 边、JPEG 0.82，base64 随 JSON 提交）发给视觉模型，按题型采用不同判分策略：

- 选择题：识别选项字母，精确比对；
- 填空题：模型判等 + 归一化（去空白、小写）精确匹配双保险。

优化策略：图片答案与文字答案**并存**（同一字段二选一提交），OCR 失败时明确提示"[图片识别失败]"并允许改用文字；判分结果与文字提交走同一套 BKT 更新链路，数据一致。

4.5 主观题 AI 评分：客观题系统判分 + 主观题模型评分

问题：解答大题、英语作文、翻译、政治简答这类主观题答案不唯一，无法用字符串比对判分。

方案：按题型分流——

- 客观题（`SINGLE_CHOICE` / `FILL_BLANK` / `CLOZE` / `READING`）由 Java 判分引擎精确或归一化比对，结果标记 `scoringMode=AUTO`；
- 主观题（`ESSAY` 解答大题 / `COMPOSITION` 作文 / `TRANSLATION` 翻译 / `SHORT_ANSWER` 简答）走 `AssessmentService.aiGrade()`：把题干、参考答案、评分要点、学生答案一起发给 GLM，要求模型返回结构化 JSON `{"score": 0~100, "correct": bool, "comment": "评语"}`。

关键设计：

- `correct` 统一按「AI 分 ≥ 60 算达标」判定，避免模型自相矛盾；
- 解析容错：浮点分四舍五入、字段缺失时降级为「待复核」；
- **降级策略**：限流（1302/1305）或解析失败时返回 `unavailable`，该题不计分、不写入错题本、不更新 BKT，前端标注「待复核」——**宁可不判，也不污染学习数据**；
- 前端结果页区分展示「AI 评分 XX 分」与系统判分，并给出 AI 评语与参考答案。

验证：实测微分方程大题答对得 100 分、胡写答案得 0 分（评语「完全偏离题意」）、政治简答 95 分、跑题作文 0 分，评分具备区分度。

4.6 MCP 工具集成：项目即 MCP Server

选型理由：MCP（Model Context Protocol）是让 AI 应用标准化调用外部工具的开放协议。把项目能力按 MCP 协议暴露，任何支持 MCP 的客户端（Cherry Studio、Claude Desktop 等）都能直接调用——这是差异化亮点，也验证了系统接口的规范性。

集成方式：

- 引入 `spring-ai-starter-mcp-server-webmvc`（Spring MVC 场景，BOM 托管坐标）；
- `KaoyanMcpTools` 类用 `@McpTool` + `@McpToolParam` **原生注解**声明 5 个工具，由 starter 的注解扫描器自动暴露；
- 端点 `/sse`（SSE 传输）与 REST 接口共存于同一应用。

踩坑与解决（重要）：最初用 Spring AI 的 `@Tool` + 手写 `ToolCallbackProvider` Bean 注册，启动即报**循环依赖**：Spring AI 的工具自动配置会收集容器内所有 `ToolCallbackProvider`，而科普/诊断服务又依赖 `ChatClient.Builder`，形成 `chatClientBuilder → toolCallbackResolver → 工具类 → service → chatClientBuilder` 死环，`@Lazy` 放在 Bean 或构造参数上都断不干净。查明 MCP starter 的注解扫描器认的是

`org.springaicommunity.mcp.annotation.McpTool` 后改用原生注解，**不注册 ToolCallbackProvider，环从源头消失**，代码也更简洁。

验证：用 Node `@modelcontextprotocol/sdk` 写了真实客户端，完整走通 `initialize → tools/list → tools/call` 协议流程，5 个工具全部正确返回。

4.7 AI 安全可控

AI 安全不是附加功能，而是贯穿设计的原则。本项目在四个层面落实：

层	措施	效果
防幻觉	事实全部来自数据库/知识库；RAG 三段式（阈值过滤 + 提示词限定 + 来源返回）；无命中明确说"没有相关信息"	模型不编造院校层次、分数线、政策
输入防护	提问截断 500 字、对话历史限量（最近 8 轮 × 200 字）、Agent 目标截断 500 字	防止超长输入稀释检索质量、撑爆 token 预算
输出可控	所有 AI 输出经规则模板兜底；失败返回友好提示而非错误堆栈	任何情况下功能可用、不 500
限流降级	每个调用点 try-catch；免费档限流时自动降级并提示用户	演示不中断
内容合规	题库全部原创（`source` 字段严格区分来源）；不收录分数线；不抓取受限站点；输出附带"以研招网和官网为准"提示	规避著作权与误导风险

五、核心功能实现

5.1 认知科普页：固定讲解 + AI 答疑双栏

设计决策：初期方案为"AI 生成讲解文章"，后改造为「**左侧固定预设文本 + 右侧独立 AI 答疑**」。理由：① 固定文本打开即读、不卡顿、不幻觉、可评审；② AI 用于"对讲解不满意时的即时追问"，职责边界清晰；③ 固定文本是纯前端静态数据，AI 故障完全不影响阅读。

实现：`scienceContent.js`（7 主题固定文本，纯静态）+ `ScienceChat.vue`（右侧 sticky 常驻对话窗，`position: sticky` 钉在视口，滚动左侧时全程可见可用）+ 后端 `/api/science/chat`（会话式 RAG，带历史与当前主题上下文）。窄屏（<920px）自动上下堆叠。

5.2 学情诊断：BKT 掌握度建模

算法：贝叶斯知识追踪（BKT）——每个知识点维护四个参数（先验 $P(L_0)$、学习率 P(T)、猜测率 P(G)、失误率 P(S)），每次作答后按贝叶斯公式更新掌握概率。纯 Java 实现于 `DiagnosisService`，可单元测试、可复现。

流程：`start`（按学科抽题，不含答案）→ `submit`（判分：选择题大小写不敏感；填空归一化；图片走 OCR）→ 逐知识点更新掌握度 → 全量诊断（未作答用先验 0.3）→ 计划生成。

多学科隔离：知识点 code 带学科前缀（`MATH-` / `ENGLISH-` / `POLITICS-`），掌握度按「学生 + 知识点」追踪，**各学科互不影响**；导入、诊断、统计全链路按前缀过滤，零 DDL 改动。

5.3 错题本：自动收录 + 导入 + 随机重做

- **自动收录**（`AssessmentService.submit` → `WrongQuestionService.recordSystemWrong`）：诊断判错即写入 `wrong_question` 表，幂等策略（同学生同题重复答错只更新错误答案、清空上次重做结果），题干冗余存储避免题库变动后丢上下文；收录失败不影响诊断主流程；

- **用户导入**（`importManual`）：题干必填，答案与备注选填，科目归一化（math/english/politics），`source=MANUAL` 与系统收录区分；
- **随机重做**（`randomRedo` + `gradeBatch`）：候选池按当前科目筛选后 `Collections.shuffle` 随机抽取（默认 5 题、上限 20）；SYSTEM 题按 `question_code` 批量回查题库拿完整题干与选项，MANUAL 题用导入时自带内容；返回结构 `RedoItem` **不含正确答案字段**——从数据结构上保证不泄露答案；交互为页面三态（列表 → 做题 → 结果），做题页布局与学情诊断一致（题卡 + 题型标签 + 选项/填空 + 底部「提交答案」），一次提交整轮由 `gradeBatch` 批量判分（逐题独立，单题异常不中断整轮）；
- **判分**（`gradeRedo`）：与学情诊断同一套规则（选择题大小写不敏感、填空去空白转小写归一化）；**答对即移出错题本**（攻克），答错保留并累计 `redo_count`、记录 `last_redo_correct`；
- **安全**：判分与移除都校验学生归属（`studentId` 不一致直接拒绝/返回 false），防止操作他人数据。

5.4 院校专业库

28 所山东院校 / 104 个热门专业，结构化字段（层次、985/211/自划线、考试科目、学制、门类）。**明确不收录分数线**（每年变动、易产生幻觉，引导学生查官网）。启动时 `CollegeDataInitializer` 自动导入；查询支持城市/层次/985/211/关键词组合筛选与统计。

5.5 AI 规划助手（Agent）

见 4.3。前端 `AgentView`：目标输入 + 示例 + 学号（选填）→ 分步执行时间线（逐条播放 + running/done/skipped + 耗时）→ 汇总建议 + 参考来源 + 降级提示。

5.6 前端工程

- **路由懒加载**：除首页外全部动态 `import()`，首屏只加载必需代码；
- **无障碍**：语义化标签、`aria-label`、对话与执行轨迹区 `aria-live="polite"` 播报、`prefers-reduced-motion`、键盘可达；
- **静态质量门禁**：`npm run check:template` 用 Vue 编译器把每个 SFC 的模板编译成渲染函数，提取 `_ctx.*` 引用与 script 定义做差集，**自动捕获"模板用了但没定义"的运行时错误**（此类错误 `vite build` 不报，曾真实发生一次：`App.vue` 状态栏绑定 `health` 而 script 定义 `healthy`）；
- **请求层**：统一 axios 封装，错误信息提取，baseURL 仅 `/api`（Vite 代理，代码不出现后端地址）。

六、测试与部署

6.1 测试策略

层	方法	覆盖
算法层	**JUnit 5 单元测试 50 个用例（`mvn test` 全绿）**	BKT 数学性质、判分归一化、计划生成、意图解析、编排与兜底、AI 对话兜底
接口层	curl/脚本对全部 REST 端点做冒烟与回归	7 控制器 31 端点
AI 链路	真实调用验证：RAG 命中与降级、Agent 两用例、MCP 客户端全协议流程	4.2/4.3/4.5
前端	生产构建 + 模板静态检查 + 页面真机打开（前后端联调）	7 视图

层	方法	覆盖
数据	题库校验器（JSON 合法性/题号重复/必填字段/题型枚举/难度/答案在选项内/知识点引用）	84 题

单元测试覆盖明细（全部不依赖 Spring 容器、数据库与大模型，纯逻辑毫秒级执行）：

测试类	用例数	验证点
`DiagnosisServiceTest`	7	BKT 方向性、0/1 吸收态、连续观测收敛、猜测率限制单次意外、更新幅度随先验变化、输出恒为合法概率
`WrongQuestionServiceTest`	9	判分规则（与诊断一致）、幂等自动记录、答对移除/答错保留、归属安全、随机抽题不回查泄露答案、导入校验
`AssessmentServiceGradeTest`	4	选择题大小写不敏感、填空归一化（全角空格）、null/空答案安全
`ChatServiceTest`	3	**模型限流/故障返回友好兜底文案（核心承诺，防回归）**、空返回兜底、正常路径
`AssessmentServiceSubmitGuardTest`	7	空/超量 answers 拒绝、超大图片拒绝、非法 base64 拒绝、**OCR 失败不计为答错**（不冤判、不污染错题本/BKT）、判错照常收录、答对不收录
`IntentToolTest`	8	院校全称/简称命中、学科关键词互含、列举式筛选词解析（山东+985/青岛+211）、科普/院校混合意图、空目标兜底、学号门控
`AgentOrchestratorTest`	6	空目标引导、**模型故障走规则兜底（核心承诺）**、模型可用返回合成内容、单工具抛错不中断、order 排序、超长目标截断
`RagServiceGuardTest`	4	输入截断上限、省略号、null 安全、零上限边界

> 测试驱动出的三个真问题：① `prior=1.0` 答错在标准 BKT 中结果恒为 1（吸收态）——已用 0.99 验证「意外」行为并写入测试文档；② 判分与归一化原为私有方法，已提为包级可见以便测试；③ **ChatService 曾是全项目唯一漏掉 try-catch 的 AI 调用点**（免费档模型一限流，AI 问答接口就返回 500）——已修复并加哨兵测试。

6.2 部署

环境要求：JDK 17 / Maven 3.9+ / MySQL 8.0+ / Node.js 18+。

启动（一键脚本 + 手动两种方式）：

```
# 后端（固定 8080：命令行参数优先级高于环境变量，防止 SERVER_PORT 变量抢端口）
mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8080
# 或双击 scripts/start-backend.bat

# 前端（5173，Vite 代理 /api → 8080）
cd frontend && npm install --registry=https://registry.npmmirror.com && npm run dev
# 或双击 scripts/start-frontend.bat
```

零配置项：启动时自动建库建表（`schema.sql`，幂等）、自动导入院校数据与三科题库（按科目幂等，老库自动补齐）、自动构建 RAG 向量库；Embedding 模型首次启动自动下载（走 hf-mirror 镜像，无需密钥）。

环境变量：仅 `ZHIPU_API_KEY` 必填；`DB_PASSWORD` 选填。密钥只存系统环境变量，代码与仓库中无任何真实密钥。

6.3 开发中定位并修复的典型问题

问题	现象	根因与修复
分词器静默回退	RAG 检索结果不相关	`tokenizer-uri` 错放在 `onnx` 节点下，回退默认 all-MiniLM 分词器与 e5 模型不匹配；移至独立节点 `transformer.tokenizer.uri`（4.2）
MCP 循环依赖	启动失败	ToolCallbackProvider 被自动收集形成死环；改用 MCP 原生 `@McpTool` 注解（4.5）
模板未定义变量	页面运行时错误（构建不报）	模板绑定名与 script 定义不一致；修复 + 静态检查脚本防复发（5.6）
端口被环境变量抢占	前端连不上后端	`SERVER_PORT` 经 Spring 宽松绑定覆盖 `server.port`；启动固定 `--server.port=8080`（命令行优先级最高）
老库存量导致新批次不导入	英语/政治数据不进库	导入器原为"表空才导入"；改为按科目幂等导入

七、总结与展望

7.1 项目总结

本项目以一个完整的"认知 → 诊断 → 计划 → 执行"闭环，验证了 **AI Native Web 应用**的设计范式：

- **AI 是核心能力而非附属功能**：RAG 答疑、Agent 编排、MCP 工具服务、OCR 多模态、BKT 智能诊断五类 AI 能力深度融入主流程；
- **事实与生成分离**：数据库与知识库存事实，模型只组织语言——这是本项目对抗幻觉、保证可信的架构级答案；
- **全链路兜底**：任何 AI 组件故障，核心功能降级可用；
- **零成本零门槛**：本地 Embedding + 本地向量库 + 免费模型档，学生项目可完整复现。

7.2 创新点（针对赛事评分）

1. **选题创新**：不做泛泛的"AI 聊天机器人"，而是切中考研人群"该不该考、考什么、怎么考"三个真问题，形成可执行路径；
2. **架构创新**：「事实/算法与模型生成分离」的分层设计 + 全链路规则兜底，系统性解决 AI 幻觉与可用性矛盾；
3. **交互创新**：Agent 执行轨迹**逐条透明播放**（意图 → 检索 → 诊断 → 计划），让用户看见 AI 的推理过程；认知科普页"固定讲解 + 常驻 AI 答疑"双栏各司其职；
4. **技术创新**：本地 ONNX RAG（零 Key 零费用）+ 自研 Agent 编排 + MCP 协议暴露 + 多模态 OCR，四类 AI 工程能力并存且互相独立可验证。

7.3 后续展望

- **题库继续扩充**：数学综合题/证明题的难度分层、英语写作的范文对比评分；
- **诊断深化**：引入遗忘曲线的间隔重复排程，错题智能复盘（薄弱点自动归因 + 针对性讲解）；
- **规划增强**：接入倒计时与每日可投入时长，生成逐日任务清单；
- **多模态扩展**：讲义拍照整页识别、手写笔记结构化。

八、团队信息

项	内容
参赛组别	B 组
成员	王传志、付赐凡、王启轩
指导教师	无
院校	山东建筑大学

附录：项目数据一览

项	数量/规格
院校库	28 所 / 104 专业（山东为主，不含分数线）
RAG 语料	29 条结构化事实
题库	共 120 题：数学 49（14 章 25 点）/ 英语 37（4 章 12 点）/ 政治 34（5 章 10 点），含选择/填空/完形/阅读/解答/作文/翻译/简答 8 种题型
REST 接口	7 个业务控制器 / 31 个端点
MCP 工具	5 个（search_colleges / get_college_detail / search_majors / ask_kaoyan_knowledge / get_study_diagnosis）
Agent 工具	5 个（Intent / CollegeSearch / Science / Diagnosis / Plan）
前端视图	7 个 + 1 个 AI 答疑组件
代码规模	后端 52 个 Java 类（+10 个测试类 66 用例）/ 前端 13 个源码模块